

DNSシンクホール構築および遠隔管理基盤の実装報告書

技術領域：Linuxネットワークインフラ / DNSセキュリティ / ゼロトラストVPN

1. 開発システム概要

本プロジェクトは、自宅内に設置したLinuxサーバー（Raspberry Pi）上に高セキュリティなDNSサーバー環境（BIND9）を構築し、特定の不要な広告・マルウェアドメインへの通信をネットワークの根幹で遮断する「DNSシンクホール」の実証実験である。

さらに、学内ネットワークやスマートフォンの回線（テザリング環境）など、厳格なファイアウォールが適用された外部環境からでも、手元の管理用PC（Zorin OS）を使って安全かつ確実にサーバーをリモートメンテナンス（SSH接続）できる「セキュアな遠隔管理基盤」の同時実装を行った。

2. システム構成・ネットワーク設計

ホスト名	OS種類	役割	ローカルIPアドレス	Tailscale IPアドレス
pi	Linux (Raspberry Pi OS)	BIND9 DNSシンクホール本体 (オンプレミスサーバー)	192.168.11.5 (固定)	100.78.116.69
keisuke-FMVA57H2BE	Zorin OS (Ubuntu ベース)	管理用クライアントPC (リモート端末)	DHCP自動割当	100.82.6.35

3. 実施フェーズとトラブルシューティング詳細

フェーズ1：オンプレミス環境の固定IP化

DNSサーバーはネットワーク全体の案内所となるため、ルーターによるIPアドレスの自動変動を防ぐ必要がある。家庭内Wi-Fiルーター（Buffalo製）の管理画面にアクセスし、ラズパイの固有のMACアドレス（ベンダーコード **DC:A6** 等で識別）に対し、ローカルIPアドレスを **192.168.11.5** に割り当てる「DHCPリース手動割当」を設定。物理再起動後も常に同じIPで起動する安定したインフラの土台を確立した。

フェーズ2：外部公開検証とプロバイダ制限（CGNAT）の切り分け

当初、一般的な「ルーターのポート変換（ポート開放：TCP 22番ポート）」を設定し、外部ネットワーク（スマホのテザリング）からサーバーへの直接SSH接続を試みた。しかし、パケットが到達せずタイムアウトエラーが発生した。

インフラエンジニア視点での原因切り分け：

宅内ローカルネットワークからは正常にSSH接続が可能であったため、ラズパイ自体の設定ミスではないと証明。外部からのパケットが遮断されていることから、契約しているインターネットプロバイダ（ISP）が「IPv4 over IPv6通信（MAP-EやDS-Lite環境）」または「CGNAT（キャリアグレードNAT）」を適用しており、ユーザー側で任意のIPv4ポートを開放できない回線仕様であると結論付けた。

フェーズ3：Tailscale（ゼロトラストVPN）による回線制限の突破

物理ルーターやISPの制限を迂回するため、代替案として暗号化オーバーレイネットワーク「**Tailscale**」を導入。ラズパイおよび管理用PCの双方にクライアントをインストールし、同一の暗号化トンネル（Tailnet）に参加させた。この際、テザリング回線によるDNS強制ブロックの干渉を受けたが、一時的に `/etc/resolv.conf` をパブリックDNS（**8.8.8.8**）に固定する名前解決のバイパス処理を実行して解決。これにより、プロバイダの制約を完全に無視し、安全なトンネルを通じた相互暗号化接続を実現した。

4. BIND9によるDNSシンクホールの実装

① ブラックホール（ゾーンファイル）の作成

広告ドメインからの問い合わせに対し、強制的に無効なIPアドレス（**0.0.0.0**）を返答する専用のゾーンファイル `/etc/bind/db.sinkhole` を作成。

```
$TTL      86400
@         IN      SOA     localhost. root.localhost. (
                        1          ; Serial
                        604800     ; Refresh
                        86400      ; Retry
                        2419200    ; Expire
                        86400 )    ; Negative Cache TTL
;
@         IN      NS      localhost.
@         IN      A       0.0.0.0
* IN      A       0.0.0.0
```

② 広告ドメインの紐付けと反映

検証用テストドメインとして **test-ad.com** を設定し、`/etc/bind/named.conf.local` に以下のルーティングを記述。構文チェック（`named-checkconf`）にてエラーがないことを確認の上、サーバーを無停止リロード（`systemctl reload named`）した。

```
zone "test-ad.com" {
    type master;
    file "/etc/bind/db.sinkhole";
};
```

5. 検証結果および動作確認エビデンス

管理用PC（Zorin OS）から、今回構築したDNSサーバー（[192.168.11.5](#)）を明示的に指定し、DNS問い合わせコマンド（[dig](#)）を実行した実際のログである。

```
; <<>> DiG 9.18.39-0ubuntu0.24.04.5-Ubuntu <<>> @192.168.11.5 test-ad.com
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29706
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; QUESTION SECTION:
test-ad.com.                IN      A

;; ANSWER SECTION:
test-ad.com.                86400   IN      A      0.0.0.0

;; SERVER: 192.168.11.5#53(192.168.11.5) (UDP)
```

■ 評価・考察（エビデンスの証明）

- **名前解決の強制遮断**：[ANSWER SECTION](#)において、対象ドメインのIPアドレスが本来のWebサーバーの数値ではなく、意図通り [0.0.0.0](#) となっており、シンクホールとしての機能が完璧に動作している。
- **権威ある回答（aaフラグ）**：ステータスフラグに刻まれた「[aa](#)（Authoritative Answer）」は、本サーバーが外部のキャッシュを流用したのではなく、自身がドメイン管理の最上流権威として主体的にこの遮断判定を下したことを意味しており、インフラとして非常に精度の高い設定が行われている。